

LIBMixer: An all-MLP Architecture for Cell Library Characterization towards Design Space Optimization

Jaeseung Lee, Sunggyu Jang, Jakang Lee, and Seokhyeong Kang

Pohang University of Science and Technology

Pohang, Republic of Korea

shkang@postech.ac.kr

ABSTRACT

Cell library characterization is a fundamental stage of electronic design automation (EDA), as it provides essential electrical models for circuit simulation and design quality assessment. However, the development of advanced nodes demands increasing computational resources and engineering efforts for characterization. We introduce **LIBMixer**, a machine learning-based framework for fast and accurate library characterization designed to enhance design space optimization. Leveraging multi-layer perceptron architectures, LIBMixer efficiently manages complex relationships between technology and electrical characteristics. It achieves a $31.5\times$ faster runtime than conventional EDA tools while enhancing alignment. Compared to state-of-the-art methods, LIBMixer targets $6.4\times$ more standard cells for both power and timing information. This scalability improvement enables the practical synthesis of IP cores, demonstrating high correlation across of power, performance, and area results. Additionally, Pareto fronts of synthesis design results with LIBMixer-inferred libraries closely match those from foundry files. Experimental results highlight the effectiveness of LIBMixer as a fast and reliable alternative for PVT analysis.

ACM Reference Format:

Jaeseung Lee, Sunggyu Jang, Jakang Lee, and Seokhyeong Kang. 2025. LIBMixer: An all-MLP Architecture for Cell Library Characterization towards Design Space Optimization. In *30th Asia and South Pacific Design Automation Conference (ASPDAC '25)*, January 20–23, 2025, Tokyo, Japan. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3658617.3697602>

1 INTRODUCTION

The field of electronic design automation (EDA) has seen significant advancements in recent years. One notable development is design space optimization (DSO), which aims to find optimal design conditions for better power, performance, and area (PPA). Since design targets typically induce changes to the layouts and configuration of standard cells, their characteristics must be reflected through library characterization. With cell library characterization, EDA tools ensure performance and verify that design results meet specifications across intended operating conditions. Efficient characterization can facilitate broader exploration of the design space, and better optimization [2–4].

Cell library characterization involves SPICE simulations to extract data on key electrical features, including delay, transition time, and power consumption under various design conditions.

However, traditional methods are becoming increasingly expensive in terms of computational costs and engineering effort due to growing complexity and varying scenarios. Various alternative methods have been proposed to mitigate the complexity of library characterization, including numerical analysis and curve fitting techniques [10, 11]. Recent research indicates that machine learning (ML)-based methodologies surpass these alternatives, as ML models adeptly capture intricate relationships and patterns within library data, resulting in more accurate and efficient characterization [6–9].

In particular, She *et al.* [7] employed linear regression to predict delay at switching times, providing accurate predictions despite its simplicity. However, the model only addressed narrow data types and transistor parameters. Jacinto *et al.* [8] utilized random forest and decision tree methods to characterize finFET technology nodes, demonstrating superior accuracy for two standard cells, *INV* and *NAND*. Despite their precision, the method struggled to expand its scope beyond the simplest cells. Cheng *et al.* [9] applied advanced architectures of heterogeneous graph attention network coupled with parasitic RC reduction to further enhance accuracy. Nonetheless, they primarily focused on improving accuracy, limiting their applicability and scope to simple standard cells for gate-level characterization, requiring graph conversion and retraining for each standard cells. Klemme *et al.* [6] approached the problem at a system-level, investigating its application to parameter optimization with ridge regression and correction by classification. However, they performed only static timing analysis on small target designs, with accuracy fluctuating by individual cell functionality.

In summary, while ML-based characterization offers advantages in addressing runtime challenges, expanding the scope to include complex standard cells and larger designs poses significant challenges. This results in a lack of comprehensive studies related to EDA tools and DSO applications. To propose a viable ML-based approach to library characterization for general design and optimization, the targeting range must be broadened to cover complex standard cells like flip-flops and latch cells. Additionally, wider patterns and various functionalities of the library data must be considered. An efficient system-level method to alternate liberty model is needed, given the increased dimensionality of the problem.

In this study, we introduce a practical approach of ML-based library characterization based on Mixer architecture, dubbed **LIBMixer**. Our framework is designed to predict library data at a system-level view, extending the target to include power and timing for a plenty range of standard cells. Compared to state-of-the-art methods [6–9], LIBMixer enhances accuracy and robustness with minimal runtime overhead. Additionally, the inferred libraries are compatible with conventional EDA tools. We observe a high correlation between design results obtained using foundry libraries and those inferred by LIBMixer across PPA metrics. Our study demonstrates that Pareto fronts of these design results with LIBMixer-inferred libraries have high correlation those from foundry files.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

ASPDAC '25, January 20–23, 2025, Tokyo, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0635-6/25/01.

<https://doi.org/10.1145/3658617.3697602>

LIBMixer presents an ML-based alternative of library characterization with significant runtime improvements. The major contributions of this study are summarized as follows:

- We introduce **LIBMixer**, a Mixer-based framework for library characterization. LIBMixer efficiently generates various type of standard cells using only PVT information, covering all essential data including timing and power.
- We demonstrate the compatibility of libraries inferred by LIBMixer with conventional EDA tools. In addition, design outcomes obtained using inferred libraries show a strong correlation with those based on foundry libraries.
- We indicate that the Pareto fronts of synthesis results using LIBMixer-inferred libraries correspond well to those from foundry files. Leveraging inferred library can be served as a efficient framework for PVT analysis.

The remainder of this paper is organized as follows, Chapter 2 describes the preliminaries of the study. Chapter 3 presents the methods of our framework. Chapter 4 describes a process and the experimental results. Finally, Chapter 5 summarizes the paper.

2 PRELIMINARIES

2.1 Cell Library Characterization

Standard cell library files are essential for enabling EDA tools to accurately model and simulate the behavior of designed circuits. These files contain critical information about standard cells, including electrical characteristics such as rise and fall delay, power consumption, constraints, and other specifications. Characterizing libraries requires extensive simulations across various behavioral patterns and design parameters, a process that becomes increasingly complex with advanced nodes.

Several studies have explored the application of ML for cell library characterization [7–9]. However, most of them focused on limited aspects of the library files, restricting characterization to the gate level and producing results for only a few cells. While gate-level approaches can enhance target accuracy, they are limited in scalability compared to system-level methodologies. The current state-of-the-art system-level models, employing ridge regression and classification, cover a broader range of cells and demonstrate the potential for design parameter optimization.

2.2 PVT Analysis

Process, voltage, and temperature (PVT) analysis is an important process for designing integrated circuit design. It evaluates entire circuit performance across various manufacturing process variations, supply voltage fluctuations, and operating temperature ranges. This comprehensive analysis ensures robust chip performance and reliability under diverse conditions. In the context of optimization, PVT analysis helps identify performance boundaries, assess trade-offs between PPA metrics, and optimize yield by ensuring designs meet specifications across different scenarios [5].

Cell libraries must be characterized across multiple PVT corners to accurately model behavior under varying conditions. This data is essential for generating timing and power models used in static timing analysis and power estimation. As semiconductor technology advances to smaller nodes, PVT analysis becomes increasingly critical due to heightened sensitivity to variations and tighter design margins. The complexity of this analysis requires advanced techniques such as ML and better sampling to balance accuracy and efficiency in the characterization process.

2.3 Design Space Optimization

Design space optimization (DSO) employs advanced algorithms to explore vast solution spaces efficiently. Unlike traditional design space exploration methods that rely heavily on manual intervention and heuristic-based searches, DSO aims to optimize multiple objectives while adhering to specified constraints. Several researches have demonstrated efficient exploration of the design space by leveraging sophisticated optimization algorithms, often incorporating elements of ML, evolutionary computation, and other AI techniques. These approaches significantly reduce the search space of the exploration process.

Fast and accurate library characterization significantly enhances DSO by providing both timely and precise electrical models. This acceleration allows for more iterations within the same turnaround time, enabling a more thorough exploration of the design space. It also improves the ability to quickly evaluate trade-offs between different design parameters, leading to optimized solutions that better meet PPA goals. By reducing the time and computational effort required for characterization, designers can focus more on innovation and optimization, ultimately achieving more efficient and effective design processes.

3 PROPOSED METHOD

The objective of the research is to characterize cell libraries using an ML-based approach, focusing on three points: (1) generating accurate libraries with shorter runtimes, (2) expanding the scope to include complex standard cells with all functionalities, and (3) constructing compatible library files for conventional EDA tools.

3.1 Overall Framework

The proposed framework comprises two stages (Figure 1): (a) the training stage and (b) the DSO stage. In the training stage, LIBMixer is trained to predict accurate library data, while the Formatter models the structure of library files. In the DSO stage, LIBMixer generates predicted data for the input target PVT conditions, which Formatter then formats into inferred libraries. After an EDA tool generates design results from inferred libraries, the design quality is produced. The Optimizer provides the next set of DSO parameters expected to yield improved design results.

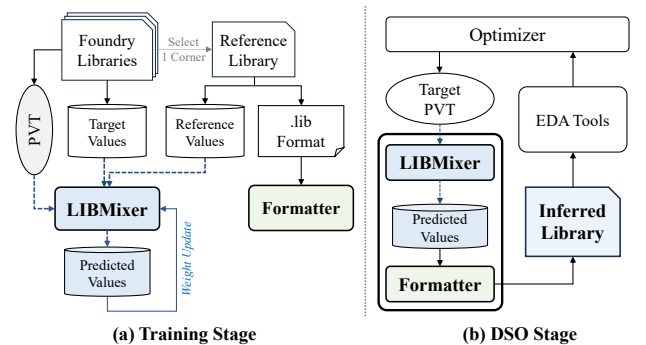


Figure 1: Overall framework of LIBMixer and the proposed DSO process. LIBMixer, a proposed ML-based model, predicts library values for target PVT conditions. The Formatter constructs library file formats (.lib) based on these values. The input and output of LIBMixer are depicted with dotted lines.

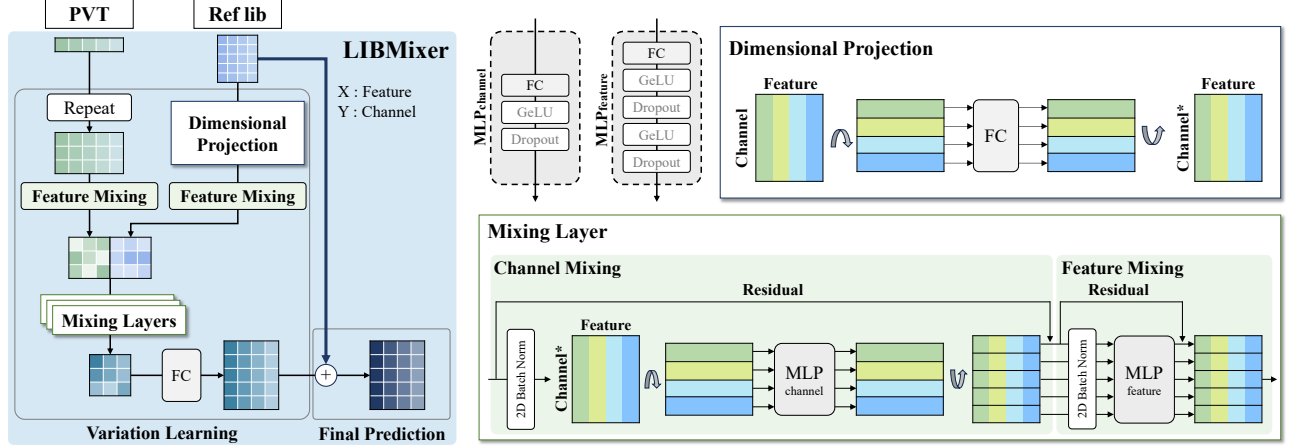


Figure 2: Structure of LIBMixer. LIBMixer predicts library values based on target PVT conditions and one reference library (Foundry-provided). It models the cell behavior by interleaving Feature Mixing and Channel Mixing layers. The Feature Mixing layer captures intra-cell characteristics, while the Channel Mixing layer models changes in specifications according to design conditions.

3.2 LIBMixer: An all-MLP Architecture for Cell Library Characterization

LIBMixer utilizes a Mixer architecture, applying MLPs across two dimensions in an alternating manner, a method proven effective in various domains [17, 18]. Since library data varies by behavioral scenarios and functionalities, the Mixer architecture addresses each information separately, then aggregates it through mixing layers. In LIBMixer, MLPs are applied to the columns (feature dimension) representing behavioral scenarios, and the rows (channel dimension) representing functionality variations. This structure efficiently handles complex non-linear relationships, all with a relatively small number of parameters. Key components are described below:

- **Mixing Layer:** Composed of two sub-layers, Channel Mixing and Feature Mixing. Mixing Layer repeatedly aggregates information gained from each sub-layers.
- **Channel Mixing:** Models dependencies among values in each library table. It includes 2D batch normalization and an MLP applied to the channel dimension. A perceptron followed by GeLU activation and dropout is used [19].
- **Feature Mixing:** Models differences between feature dimensions. Following Transformer and time-series mixer [18, 19], two-layer MLPs are used to learn complex feature relations, containing GeLU activation function and dropout layers.
- **Residual Connection:** Applied before each layer, this allows the network to be deeper while enabling it to ignore unnecessary mixing operations if needed.
- **Dimensional Projection:** A fully-connected layer applied to the channel domain that learns library data patterns and maps the input channel size to a hidden channel size.

As illustrated in Figure 2, the two Feature Mixing layers align the PVT parameters with the reference library values, ensuring dimensional consistency, and multiple mixing layers then combine these characteristics. The output of the mixing layers is trained to capture any discrepancies from the reference values, and the final prediction is obtained by adding these stored reference values.

3.2.1 Differences from Typical Mixer-based Models. Unlike typical Mixer-based models, LIBMixer ensures that the output of the mixing layers matches the size of the input reference library tables. This is followed by an additional residual connection, which enhances the

precision of data distributions. The structure is designed to handle high-dimensional problems and provides superior generalization with a small number of training samples.

Moreover, LIBMixer manages the variable shapes of library tables by distinguishing between their formats. In addition to the 2D tables used for delay, transition time, power, and timing constraints, 1D tables capture features such as passive power varying by pin transitions. Leakage power, determined by transistor features, is represented by a single value. LIBMixer adapts its channel values according to the number of elements in each target feature and trains these elements separately. The library files are then concatenated in the correct order by the Formatter.

3.2.2 Training Objective. Similar to previous works [6, 9], we use a mean square error (MSE) loss function defined as:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (\mathcal{F}(x_i; \theta) - y_i)^2, \quad (1)$$

where N is the number of training samples in a batch, x_i and y_i are the i -th input pattern and corresponding target library value, respectively. θ represents the model parameters.

4 EXPERIMENT SETUP AND RESULTS

4.1 Experimental Setup

The experiments were conducted on a computing system with a 2.8 GHz AMD EPYC 7402P CPU, 256 GB RAM, and an NVIDIA Titan RTX GPU. The library dataset was generated employing the *BSIM-CMG model* [21] and *Synopsys SiliconSmart* [22]. Synthesis data was generated using the *Synopsys Library Compiler* [23] and *Synopsys Design Compiler* [24], with designs from *OpenCores* [25] and *RocketChip* [26]. The specifications of the circuit designs are presented in Table 1.

Table 1: Design specifications

Bench Name	Target Clock [ns]	# of Instances	Design Time [s]
<i>ibex</i>	2500	15k	80
<i>des3</i>	1000	60k	120
<i>swerv</i>	600	61k	400
<i>ldpc decoder</i>	1000	42k	1000
<i>rocket core</i>	600	700k	9000

Table 2: Target standard cells

Types	# of Cells	Prefix	Sizes
AO	42	AO2	1x1/2, 11x2, 2x1/2, 21x1/2, 22x2
		AO3	1x2, 2x1/2, 22x2, 3x2, 31x1/2, 32x1/2, 33x1/2
		AOI2	1x1/xp33/xp5, 11x1/xp5, 2xp5/33, 21x1/xp5, 22xp33/x1
		AOI3	1/11/2/21/22/3/31/32/33xp33, 1xp67
		Others	A2O1A1xp33 A2O1A1O1xp25
INVBUF	37	BUF	x2/3/4/4f/5/6f/8/10/12/12f/16f/24 / HB1/2/3/4xp67
		CKINVDC	x5p33/6p67/8/9p33/10/11/12/14/16/20
		INV	x1/2/3/4/5/6/8/11/p33/p67
OA	34	OA2	1x2, 11x2, 2x2, 21x2, 22x2
		OA3	1x2, 3x2, 31x1/2, 32x1/2, 33x1/2
		OA12	1x1/p5/p33, 11xp5, 2x1/p5/p33, 21xp5, 22xp33
		OA13	1/11/2/21/22/3/31/32/33xp33, 1xp67
		Others	O2A1O1xp33/p5
SEQ	33	D F/F	DFFASRHNx1, DFFHQNx1/2/3/4 DFFLQNx1/2/3/4
		D Latch	DHLx1/2/3 DLLx1/2/3
		ICG	x1/2/2p67DC/3/4/4DC/5/5p33DC/6p67DC/8DC
		Scan D F/F	SDFHx1/2/3/4 SDFLx1/2/3/4
SIMPLE	52	AND	2x2/4/6, 3x1/2/4, 4x1/2, 5x1/2
		NAND	2x1/1p5/2/p33/p5/p67, 3x1/2/p33, 4xp25/p75, 5xp2
		NOR	2x1/1p5/2/p33/p67, 3x1/2/p25/p33/p75, 5xp2
		OR	2x2/4/6, 3x1/2/4, 4x1/2, 5x1/2
		Others	XNOR2x1/2/5 MAJ1x2/3/p5 XOR2x1/2/p5
Total			198

Table 3: DSO parameters and specifications

Symbol [*]	Unit	Min	Max	Step Size	# of Comb.
Φ_g	eV	4.250	4.450	0.005	41
V_{dd}	V	0.55	0.75	0.05	5
T	°C	[-50, 0, 25, 75, 125]			5

Total number of combinations: 1025
(Train / Validation / Inference : 800 / 180 / 45)

^{*} Symbol Φ_g represents the gate work function of the FinFET, V stands for the supply voltage, and T states the temperature of the design. Comb. stands combinations.

4.1.1 Target libraries. To ensure wide applicability to EDA tools, we selected 198 cells of the FinFET ASAP7 process design kit (PDK) [20] while previous methods targeted 31 cells (6.4×) from older PDKs. Detailed information about the target standard cells is provided in Table 2. Additionally, the parameter space was defined referring FinFET PDK PVT corners to cover extreme conditions, as shown in Table 3. In our approach, gate work function tuning was employed to adjust the threshold voltage due to potential effects like random dopant fluctuation. The *BSIM-CMG* model was used to enhance stability and reliability by accounting for physical effects.

4.1.2 Dataset. We chose a test dataset covering extreme PVT corners with $\Phi_g \in \{4.25, 4.30, 4.35, 4.40, 4.45\}$, $V \in \{0.55, 0.65, 0.75\}$ and $T \in \{-50, 25, 125\}$, which were excluded from the training dataset, as shown in Table 3. From the remaining 980 combinations, 800 were used for training and 180 for validation.

4.1.3 Model. We compared the performance of **LIBMixer** with the state-of-the-art **Ridge** model [6], which is currently the only method offering system-level characterization. LIBMixer incorporates residual connections and log normalization during preprocessing, in contrast to the linear min-max normalization used in previous work. Since library data is structured in the form of a look-up table, where the spacing of the input values progressively widens, it effectively follows a power distribution. Log normalization helps to address this by transforming the data into a more uniform scale, simplifying sequences and compresses the data range and improving coverage of extreme PVT conditions. For a comprehensive comparison, we also propose the **Ridge+** model, which applies log normalization and residual connections to the Ridge model. The training and validation times per epoch ranged from 1.4 to 681.6 seconds for Ridge, 1.9 to 689.5 seconds for Ridge+, and 14.0 to 770.2 seconds for LIBMixer, depending on the data type.

Table 4: Model and training parameters

Parameters	Values
hidden channel / feature	8 / 64
number of mixer blocks	3
optimizer / train epoch	ADAM / 1000
learning rate (LR) / weight decay	1e-3 / 1e-5
LR scheduler / step / gamma	stepLR / 100 / 0.8

To measure the performance of ML estimators, we rely on several key metrics computed from the library data, including mean absolute error (MAE), mean absolute percentage error (MAPE), and standard error (SE). Other parameters are listed in Table 4.

4.2 Accuracy of Library Prediction

We conducted experiments to evaluate the performance of LIBMixer in predicting library values and verified the accuracy of the trained model. The results are presented in Table 5. During the training phase, Ridge, Ridge+, and LIBMixer exhibited comparable MSE values. However, Ridge's test predictions showed approximately 2.23× higher MSE compared to LIBMixer. While Ridge performed well in generating simple library tables, its accuracy declined when applied to larger and more diverse sets of cells, as seen in our test set. LIBMixer achieved an overall MAPE of 0.05% to 1.42%, with less than 1.7% runtime overhead compared to Ridge. Its robustness and generalization capabilities exceeded those of Ridge, making it more effective for handling complex and diverse cell libraries.

Additionally, LIBMixer was on average 31.5× faster in generating libraries, taking only 111.0 seconds compared to the about 3,500 seconds required by the golden tool. Figure 3 presents the scatter plot of inference results, illustrating the relationship between the golden values (x-axis) and predicted values (y-axis) for different data types. The correlation between golden and predicted values improves from top to bottom, highlighting LIBMixer's accuracy.

4.2.1 Ablation Study on Effectiveness of Log Normalization and Additional Residual Connection. Ridge+ model, which incorporates two modifications, generally achieves better accuracy than the original Ridge model, reducing the MAPE by up to 8.21%. However, this modification adversely affects leakage power data. The lack of improvement in leakage power prediction is due to the

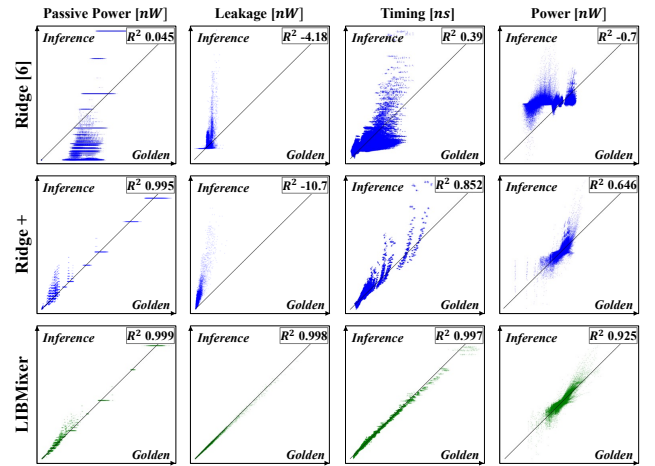


Figure 3: Scatter plot of inference results of each model. LIBMixer outperforms the highest correlation between golden and predicted values compared to other ML-based approaches.

Table 5: Comparative analysis with state-of-the-art machine learning-based library characterization

Metric	Golden	Passive Power [nW]			Leakage [nW]			Timing [ns]			Power [nW]		
		Ridge [6]	Ridge+	Ours	Ridge [6]	Ridge+	Ours	Ridge [6]	Ridge+	Ours	Ridge [6]	Ridge+	Ours
Train MSE	0.00E+00	3.20E-04	3.13E-04	2.27E-04	2.33E-03	2.25E-03	2.04E-03	31.0E-04	14.7E-04	7.40E-04	3.70E-03	4.40E-03	1.69E-03
MAE	0.00E+00	7.54E+00	8.03E-02	3.39E-02	3.11E+04	1.89E+04	8.76E+02	5.77E+01	2.07E+01	3.25E+00	1.06E+00	1.71E-01	6.75E-02
MAPE [%]	0.00	7.05	1.19	0.69	1.14	1.33	0.05	1.77	0.24	0.11	10.39	2.18	1.42
R2	1.000	0.0449	0.9945	0.9989	-4.180	-10.704	0.9980	0.3870	0.8516	0.9967	-0.078	0.6465	0.9253
Runtime	3,500 ± 1,000	27.346	27.313	27.645	27.736	27.713	27.682	27.363	27.761	27.824	27.359	27.723	27.784

The best results are highlighted in **bold** for each metric.

Table 6: Logic synthesis results for power and area

Bench	Metric	Dynamic Power [μ W]				Leakage Power [μ W]				Area [μm^2]			
		Golden	Ridge [6]	Ridge+	Ours	Golden	Ridge [6]	Ridge+	Ours	Golden	Ridge [6]	Ridge+	Ours
ibex core	Mean	927.63	169003.26	1031.60	928.77	148.53	831.53	50.81	149.59	1783.68	3085.92	1785.85	1780.65
	MAE	-	168075.63	928.77	21.12	-	683.00	149.59	2.90	-	1302.24	1780.65	9.79
	SE	-	2132.80	36.07	2.86	-	99.38	65.39	1.45	-	60.26	4.55	1.29
	MAPE [%]	-	19200.75	24.54	2.41	-	6764624.63	40562.65	3.00	-	73.06	1.32	0.54
des3	Mean	17897.06	1751695.45	15827.31	17293.59	625.12	3619.58	174.39	653.33	6409.15	10446.89	6579.95	6410.06
	MAE	-	1733798.40	17293.59	1873.02	-	2994.46	653.33	29.05	-	4037.74	6410.06	22.33
	SE	-	32682.81	1086.28	215.12	-	432.40	288.00	16.50	-	97.06	4.90	3.54
	MAPE [%]	-	10299.34	18.87	11.59	-	2395032.52	30837.54	5.51	-	63.00	3.87	0.35
swerv	Mean	2482.21	601281.04	2557.43	2490.78	1025.84	4298.87	320.84	1045.04	12069.23	20684.03	12065.36	12047.40
	MAE	-	598798.83	2490.78	168.44	-	3290.91	1045.04	28.13	-	8614.80	12047.40	32.78
	SE	-	7927.94	130.68	26.82	-	508.11	459.57	14.89	-	607.79	2.98	3.98
	MAPE [%]	-	25845.50	8.38	6.66	-	380102.68	18730.34	2.70	-	71.35	0.26	0.27
ldpc	Mean	8240.45	429495.68	7718.25	8233.07	611.29	1281.80	189.33	613.09	5687.79	7252.59	5390.50	5731.26
	MAE	-	421255.22	1008.72	820.08	-	964.44	423.53	11.14	-	1601.17	339.82	91.38
	SE	-	7577.89	132.52	158.01	-	145.17	189.48	4.96	-	173.69	34.36	18.81
	MAPE [%]	-	5643.57	15.28	9.95	-	328164.90	23076.28	4.43	-	28.51	5.91	1.60
rocket core	Mean	32518.45	4364300.00	31529.97	32438.07	10538.28	25188.10	3100.36	10470.62	105738.26	143488.95	104766.97	105678.41
	MAE	-	4331781.55	11680.01	1215.61	-	18789.08	7479.72	255.81	-	37750.69	1577.32	816.01
	SE	-	55333.70	1738.80	203.59	-	2656.14	3411.89	104.60	-	2196.57	214.71	85.82
	MAPE [%]	-	16602.46	39.49	4.09	-	2910.60	88.61	4.43	-	35.71	1.49	0.77

The best results are highlighted in **bold** for each metric.

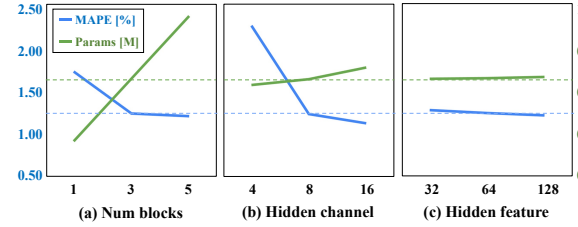


Figure 4: Ablation of model parameter and accuracy, with Power data plotted. The blue line and green line represent MAPE [%] and the total number of parameters [Million], respectively.

zero-dimensional nature of leakage power values, where log normalization has less effect. A comparison between the Ridge+ model and LIBMixer reveals that the proposed Mixer-based architecture is more effective, as it can generate highly correlated data and reduce the MAPE by up to 1.08%. Considering both accuracy and runtime, the proposed LIBMixer outperformed all other methods.

4.2.2 Ablation Study on Sensitivity of Model Hyper Parameters. An ablation study was conducted on the structural parameters of LIBMixer, including the number of mixer blocks and the sizes of hidden channels and features. As depicted in Figure 4, increasing each hyperparameter led to an increase in the number of model parameters and a reduction in error. Nevertheless, the improvement becomes negligible after three blocks due to the residual connection of Mixing-layer, which prevents overfitting and ignores unnecessary operations.

Increasing the size of hidden channels or hidden features improved the performance with only a slight increase in model complexity. Notably, expanding the hidden channels yielded the best result, with the number of model parameters increased only 1.11%. The study highlights the effectiveness of the Mixer structure and suggests that enlarging hidden channels or features after a few stacking layers can contribute to reducing the MAPE.

4.3 Compatibility of Inferred Library

We evaluated the compatibility of LIBMixer by comparing post-synthesis design results from inferred libraries to those generated using foundry libraries. The basic objective of synthesis tool is to achieve timing constraints while minimizing both power and area. Therefore, analyzing power and area results in conjunction with timing optimization is crucial. Results for power, area, and worst slack (WS) are shown in Table 6, 7. Table 6 presents a comparison of power and area results, while Table 7 provides the performance results, indicated by worst slack (WS) comparison. In our test set design results, libraries inferred by LIBMixer demonstrate the same violation cases as foundry libraries while achieving high accuracy in evaluation metrics. For power and area, the overall mean is closest to the golden results, with errors of 0.83%, 1.61%, 0.24% for dynamic power, leakage power, and area, respectively. The average MAPE

Table 7: Logic synthesis results for worst slack

Bench	Metric	Worst Slack [ns]			
		Golden	Ridge [6]	Ridge+	Ours
ibex core	Mean	-87.08	-6707.31	-19.59	-92.98
	MAE	-	6620.23	92.99	8.96
	SE	-	240.34	47.05	3.16
	# V	6	44	3	6
des3	Mean	60.44	-2233.24	39.87	63.47
	MAE	-	2293.68	63.47	64.42
	SE	-	80.31	12.15	10.89
	# V	0	44	0	0
swerv	Mean	446.6	411.8	441.9	446.7
	MAE	-	49.85	446.68	7.02
	SE	-	4.69	3.43	1.49
	# V	0	0	0	0
ldpc	Mean	-8.99	-2005.18	1.64	-10.39
	MAE	-	1996.19	11.59	3.78
	SE	-	93.44	6.14	1.56
	# V	3	44	0	3
rocket core	Mean	101.49	-104.95	157.09	142.02
	MAE	-	206.44	108.53	62.61
	SE	-	22.76	14.66	8.75
	# V	0	25	0	0

The best results are highlighted in **bold** for each metric. # V stands the number of violated designs.

for dynamic power, leakage power, and area is 6.94%, 4.04%, 0.71%, respectively. Additionally, LIBMixer consistently achieves lower MAE and SE across various benchmarks, indicating its robustness and reliability in library characterization.

Conversely, Ridge models show significant discrepancy, which is primarily attributed to the misprediction of complex standard cells. Specifically, for D flip-flop cells, the average MAPE for Ridge-generated cell timing and power is 23.8% and 88.6%, respectively, while LIBMixer achieves much lower errors of 0.12% and 2.07%, respectively. D Flip-flops are critical instances, with their minimum proportion ranging from 4.8% in *ldpc* to 23.5% in *swerv* and 30.1% in *rocket core*. For the *swerv* benchmark, the MAPE of power and area is the highest, and even for *rocket core*, the design constraints are easily violated. Previous studies performed only static timing analysis and timing prediction for combinational logic cells, excluding flip-flops, which is why they did not encounter this error. This analysis highlights the importance of extending the scope of library characterization to include sequential cells, which is essential for producing accurate design results.

The compatibility offers opportunities for usages of ML-based library characterization in PVT analysis. In PVT analysis, it is necessary to test the design with various PVT libraries after the design to ensure its validity under specific conditions. In this step, LIBMixer can predict results closely equivalent to using actual libraries.

4.4 Design Space Optimization

To assess the performance of LIBMixer in DSO, we synthesized opensource benchmark circuits and compared the results of inferred libraries with those generated using foundry libraries. Specifically, we focused on two key design metrics: (a) power vs. performance and (b) power vs. area, which are critical for evaluating the quality of target circuit designs.

In our experiments, we analyzed the power-performance Pareto front for two designs: (1) *ibex core* and (2) *rocket core*. Figure 5 shows the Pareto front analyses, illustrating the trade-offs between power and performance (*p-t*) and between power and area (*p-a*). In the performance vs. power graphs ((a) and (c) in Figure 5), the x-axis represents the inverted WS, where lower values correspond to higher performance levels, while the y-axis indicates total power, with lower values reflecting lower power consumption. In the area vs. power graphs ((b) and (d) in Figure 5), the x-axis represents area, where smaller values signify more compact designs, and the y-axis shows total power consumption.

Our results demonstrate that the Pareto fronts of generated by LIBMixer are closely aligned with those from the foundry libraries, with the Euclidean distance between corresponding points on the Pareto fronts being less than 1.51% for power-performance and 3.1% for power-area trade-offs. This high level of accuracy underscores the capability of LIBMixer to generate library data that effectively supports DSO processes without compromising design quality.

In addition, one of LIBMixer's key advantages is its significant reduction in runtime. Foundry libraries often require extensive time for cell characterization and testing across multiple PVT corners, which can be a bottleneck. Faster inference mechanism accelerates this process, allowing designers to rapidly generate library data for various PVT conditions. This is particularly valuable for identifying and validating critical PVT conditions efficiently.

The combination of LIBMixer's accurate Pareto front alignment and substantial runtime reduction demonstrates the clear potential

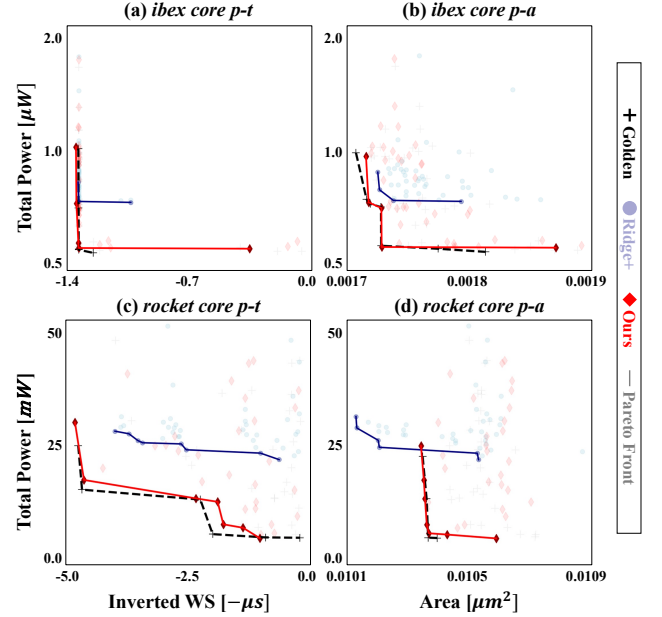


Figure 5: Pareto points and fronts of post-synthesis PPA obtained from foundry libraries, Ridge+ and LIBMixer inferred libraries. Design results with LIBMixer inferred library demonstrates close alignment Pareto fronts to those of using the foundry library.

of machine learning techniques to improve DSO processes in EDA. By enabling faster design space exploration without sacrificing reliability, LIBMixer offers a scalable and efficient solution for ML-based library characterization in DSO applications.

5 CONCLUSION

In this study, we introduce **LIBMixer**, an ML-based approach to library characterization. LIBMixer effectively extended the scope of ML-based characterization across vast standard cells and electrical behaviors including timing and power, with a 31.5× faster runtime than conventional methods. Without relying on physical characteristics, LIBMixer can generate inferred libraries only based on PVT information that are perfectly compatible with EDA tools. Our results show that LIBMixer achieves superior performance compared to other machine learning baselines. Moreover, we validate the compatibility and performance of LIBMixer inferred libraries in post-synthesis design results, demonstrating comparable results to foundry libraries across power, area, and timing metrics. Lastly, we illustrated the applicability of LIBMixer through the design space optimization, demonstrating its ability to achieve high correlation with traditional methods but with significantly reduced runtime. LIBMixer inferred library can serve as a quick foundation for PVT selection and analysis. Future work will focus on further optimization and automating the PVT corner selection process.

ACKNOWLEDGMENTS

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2023-00222085, Development of memory module and memory compiler for non-volatile PIM) and Samsung Electronics Co., Ltd. The EDA Tool was supported by the IC Design Education Center.

REFERENCES

- [1] V. Moroz *et al.*, “DTCO Launches Moore’s Law Over the Feature Scaling Wall,” *Proc. IEDM*, 2020.
- [2] S. Choi *et al.*, “PROBE3.0: A Systematic Framework for Design-Technology Pathfinding with Improved Design Enablement,” *IEEE Transactions on CAD*, 2021.
- [3] CK. Cheng *et al.*, “Complementary-FET (CFET) standard cell synthesis framework for design and system technology co-optimization using SMT,” *Proc. IEEE Transactions on VLSI*, 2021.
- [4] J. Lee Park *et al.*, “Multi-Source Transfer Learning for Design Technology Co-Optimization,” *Proc. ISLPED*, 2023.
- [5] Y. Greg *et al.*, “The past present and future of design-technology co-optimization,” *Proc. CICC*, 2013.
- [6] F. Klemme *et al.*, “Cell Library Characterization using Machine Learning for Design Technology Co-Optimization,” *Proc. ICCAD*, 2020.
- [7] Y. She *et al.*, “Standard Cell Library Characterization of 28nm Process Based on Machine Learning,” *Proc. DEStech CSE*, 2017.
- [8] G. Jacinto *et al.*, “Fast and Low-Error Prediction of Logic Gate Cell Characterization,” *Proc. ICECS*, 2023.
- [9] X. Cheng *et al.*, “Heterogeneous Graph Attention Network Based Statistical Timing Library Characterization with Parasitic RC Reduction,” *Proc. ASP-DAC*, 2024.
- [10] K. Charafeddine and F. Ouardi, “Fast timing characterization of cells in standard cell library design based on curve fitting,” *Proc. WITS*, 2017.
- [11] L. Yu *et al.*, “Statistical library characterization using belief propagation across multiple technology nodes,” *Proc. DATE*, 2015.
- [12] D. Chicco *et al.*, “The coefficient of determination R-squared is more informative than SMAPE, MAE, MAPE, MSE and RMSE in regression analysis evaluation,” *PeerJ Computer Science*, 2021.
- [13] W. Chang *et al.*, “Modifying Threshold Voltages to n- and p- Type FinFETs by Work Function Metal Stacks,” *IEEE Open Journal of Nanotechnology*, 2021.
- [14] K. Akiyama *et al.*, “Threshold Voltage Tuning of n- and p-Type FinFETs by Work Function Metal Stacks,” *IEEE Electron Device Letters*, 2020.
- [15] V. De *et al.*, “Near threshold voltage (NTV) computing: Computing in the dark silicon era,” *IEEE Design & Test*, 2017.
- [16] J. Jung *et al.*, “METRICS2.1 and Flow Tuning in the IEEE CEDA Robust Design Flow and OpenROAD,” *Proc. ICCAD*, 2021.
- [17] O. Tolstikhin *et al.*, “Mlp-mixer: An all-mlp architecture for vision,” *Proc. NeurIPS*, 2021.
- [18] S. Chen *et al.*, “Tsmixer: An all-mlp architecture for time series forecasting,” *arXiv preprint*, arXiv:2303.06053, 2023.
- [19] A. Zeng *et al.*, “Are transformers effective for time series forecasting?,” *Proc. AAAI*, 2023.
- [20] L. Clark *et al.*, “ASAP7: A 7-nm finFET predictive process design kit,” *Proc. Microelectronics Journal*, 2016.
- [21] S. Khandelwal *et al.*, “BSIM-CMG 110.0.0: Multi-gate MOSFET compact model: technical manual,” 2014.
- [22] Synopsys Inc, “SiliconSmart ACE 2021,” in *Software*, 2021.
- [23] Synopsys Inc, “Library Compiler 2018,” in *Software*, 2018.
- [24] Synopsys Inc, “Design Compiler 2018,” in *Software*, 2018.
- [25] OpenCores: “Open Source IP-Cores,” <http://www.opencores.org>.
- [26] A. Krste *et al.*, “The rocket chip generator,” EECS Department, University of California, Berkeley, Tech., 2016.
- [27] A. Paszke, *et al.* “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *Proc. NeurIPS*, 2019.